

☁ Serverless — Easy KCNA Notes

Since you're learning **KCNA as a beginner**, let's understand Serverless step by step.

1. What is Serverless?

Serverless is a cloud computing model where you run your application without managing the underlying servers yourself.

⚠ **Important:** Serverless does **NOT** mean there are no servers.

There are still servers, but the **cloud provider manages them for you.**

🧠 **Easy memory:**

Serverless = You manage the code, cloud provider manages the servers.

2. Traditional Server vs Serverless

💻 Traditional

You are responsible for things like:

You



Application



Operating System



Server



Networking



Infrastructure

You may need to:

- Create servers

- Configure servers
 - Install OS/software
 - Manage capacity
 - Patch systems
 - Scale infrastructure
-

Serverless

The cloud provider handles the infrastructure:

You



Your Code



Cloud Provider



Servers



Infrastructure

You mainly focus on your **application/code**.

3. Real-Life Example

Imagine you want to sell pizza.

Traditional approach

You buy:

-  Restaurant
-  Oven

- 🍽️ Tables
- 👨🍳 Equipment

You manage everything.

Serverless approach 🍕

You use a shared kitchen.

Someone else manages:

- Building
- Electricity
- Equipment
- Maintenance

You mainly focus on **making the pizza**.

That's similar to serverless.

4. How Serverless Works

A common serverless model is **Function as a Service (FaaS)**.

You write a small function:

User sends request



Function runs



Response returned

For example:

User



"Calculate total"



Function



Total = \$50

The cloud provider automatically handles the servers required to run that function.

5. Serverless and Events ★

Serverless functions are often **event-driven**.

That means something happens → function runs.

Example:

User uploads image



Event



Function runs



Resize image

Other examples:

HTTP Request



Function

File Upload



Function

Database Event



Function

Message Arrives



Function

6. Serverless Scaling

One major advantage is automatic scaling.

Suppose:

10 requests



Function runs

Suddenly:

10,000 requests



Serverless platform



Automatically provides more capacity

You don't normally have to manually create and configure individual servers for those requests.

7. Pay for Usage

Serverless platforms commonly use a **usage-based pricing model**.

Instead of continuously paying for an idle server, you generally pay based on things such as:

- Number of requests
- Execution time

- Memory/resources used

For example:

No requests



Little or no function execution

More requests



More execution



Higher usage

The exact pricing depends on the provider and service.

8. Advantages of Serverless

1. Less server management

You don't have to manage the underlying servers yourself.

2. Automatic scaling

The platform can automatically scale according to demand.

3. Faster development

Developers can focus more on application logic.

4. Usage-based billing

You commonly pay according to usage rather than maintaining constantly running servers.

5. Event-driven

Functions can automatically respond to events.

9. Disadvantages of Serverless

Serverless isn't perfect.

1. Cold starts

A function that hasn't been running recently may take some additional time to start.

Request



Function not ready



Start function



Execute

This initial delay is called a **cold start**.

2. Vendor lock-in

Your application may become closely tied to a particular cloud provider's serverless services.

3. Execution limits

Serverless functions often have limits such as:

- Maximum execution time
- Memory limits
- Package/deployment limits

4. Less infrastructure control

You don't have the same level of control over the underlying servers as you would with your own VMs or physical servers.

10. Serverless vs Containers ★★ ★

This is important because you're learning **Docker and Kubernetes**.

Containers

You package your application into a container:

Docker



Container



You manage more of the application environment

Kubernetes can manage those containers:

Kubernetes



Pods



Containers

Serverless

Your Code



Serverless Platform



Cloud manages infrastructure

Easy difference:

Containers → You have more control.

Serverless → Cloud provider manages more infrastructure for you.

11. Serverless vs Kubernetes

They solve somewhat different problems.

Kubernetes

Kubernetes manages containerized workloads:

Kubernetes



Pods



Containers



Applications

You generally manage/configure things such as:

- Pods
- Deployments
- Services
- Nodes
- Networking
- Resources

Serverless

Code



Serverless Platform



Cloud handles infrastructure

You focus mainly on your application code and configuration.

12. Where is Serverless Useful?

Serverless is particularly useful for:

-  APIs
-  Image processing
-  Event processing

-  Notifications
-  Data processing
-  Short-running tasks
-  Backend functions for applications

Example:

User uploads photo



Cloud storage



Event



Serverless function



Resize photo



Save result

KCNA Revision Notes

Serverless

-  Cloud computing model.
- **Servers still exist.**
- Cloud provider manages the underlying infrastructure.
- Developer focuses mainly on application code.
- Often uses **Functions as a Service (FaaS)**.
- Usually **event-driven**.

- Can automatically scale according to demand.
- Commonly uses usage-based billing.
- **Cold starts** can cause initial latency.
- Can have execution/resource limits.
- May create **vendor lock-in**.

★ One-line definition

Serverless is a cloud computing model where the cloud provider manages the underlying infrastructure and automatically runs and scales your application code based on demand.

🧠 Super Easy Memory

Traditional



YOU manage servers 🖥️

Containers



YOU manage containers 📁

Serverless



CLOUD manages servers ☁️

Remember:

Serverless ≠ No servers.

It means **you don't have to manage the servers yourself.**